

DEEP LEARNING, TOLD SIMPLY

Book 3 of 5

Guessing “How Much”

Take away the cliff and one neuron can say a number. Give it a score for being wrong, and it can walk downhill to the best answer. This is the book where the maths of learning begins — and every step is shown.

Inside: a neuron with no threshold · regression, or answering “how much” · the squared-error loss and its bowl · the slope of the bowl, derived line by line · batch gradient descent, run by hand on four cars · learning rates that crawl, halve, jump and explode · the real thing: fuel mileage of 392 cars (UCI Auto MPG) · loss curves, predictions, and a neuron in twenty-five lines



$$w \leftarrow w - \eta \cdot dL/dw$$

Deep Learning (DNN) · a five-book story

Prof. Saurabh

Where we are in the story

LAST TIME

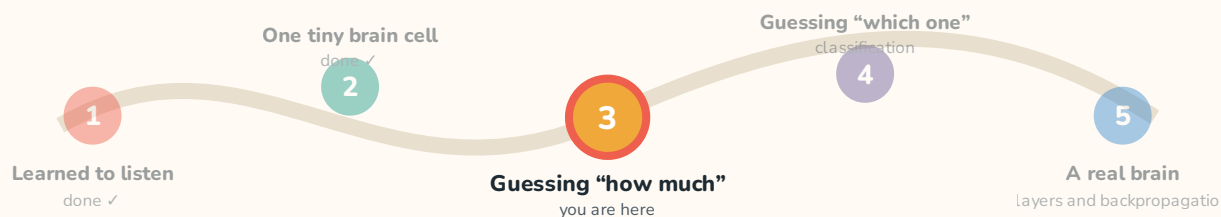
We opened Nova's head and found the **perceptron**: multiply inputs by weights, add the bias, and fire if the total is zero or more. It is a straight wall with a side. Rosenblatt's rule taught it NOT, AND and OR with a pencil, and Rosenblatt's promise said it will always find a wall when one exists.

But two things were wrong with it. Its answer was a cliff — 0 or 1, never “almost” — so its learning rule was deaf to *how* wrong a guess was. And XOR showed that one wall is not enough; we need layers, and the rule cannot train them.

THIS TIME

We fix the first problem, and in doing so we build the tool that will fix the second. Take the cliff away: let the neuron's output be the total itself, a plain number. Now it can answer “**how much**” — a car's fuel mileage, say.

Then give it a proper score for being wrong (the squared error from Book 1), and discover that the score is a **bowl** over the knobs. Learning becomes walking downhill. We derive the slope of the bowl step by step, walk down it by hand on four cars, and then let the code walk down it on 392 real ones. That walk is **gradient descent**, and it trains everything from here to the end of the course.



What you need

Pencil and paper again. The only new maths is one idea: *if you nudge a knob a tiny bit, how much does the score change?* We build that idea from scratch in Chapter 4, with numbers, before we give it its name. If you have seen derivatives before, you will recognise it; if you have not, you will still be fine. The companion notebook (`Book3_Regression_From_Scratch.ipynb`) reproduces every number in this book.

The boxes you will meet

Same as Books 1 and 2. **Story** · **Big idea** · **Think of it like...** · **Stop and think** · **Our answer** · **Try it yourself** · **Careful** · **Spark** · **Words to know**. New in this book: **derivation boxes** with numbered steps — every line of algebra, nothing skipped.

1

A neuron that says a number

Remove the cliff from the perceptron and it stops shouting yes or no. It starts answering “how much?”

STORY

Asha’s uncle is buying a used car. He has narrowed it down to three, and for each one he knows the weight and the engine’s horsepower — but not the fuel mileage, which is the thing he actually cares about. “You’re the AI student,” he says. “Tell me how many miles per gallon each one will give.” Asha realises this is not a yes/no question. Nothing in Book 2 can answer it. The perceptron would only say “good mileage” or “bad mileage”; her uncle wants a *number*.

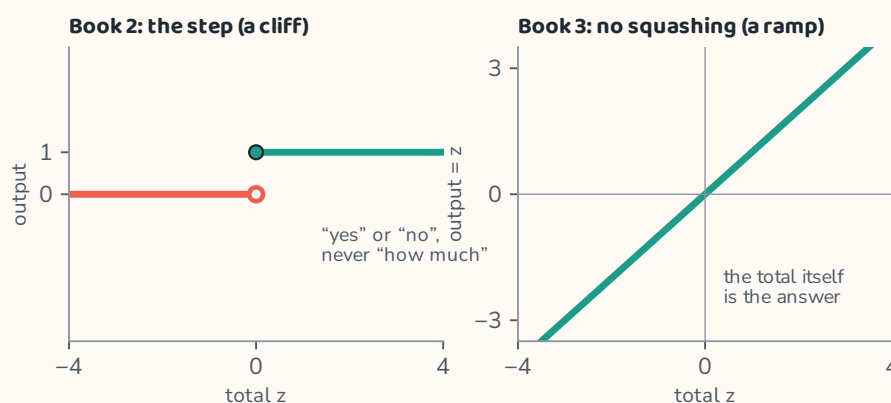
The smallest possible change

Look at the perceptron again: $z = w_1x_1 + w_2x_2 + b$, and then *if $z \geq 0$, say 1, else say 0*. That last step is a cliff. Everything to the left of zero is flattened to 0, everything to the right to 1. All the information in z — whether it was 0.1 or 100 — is thrown away.

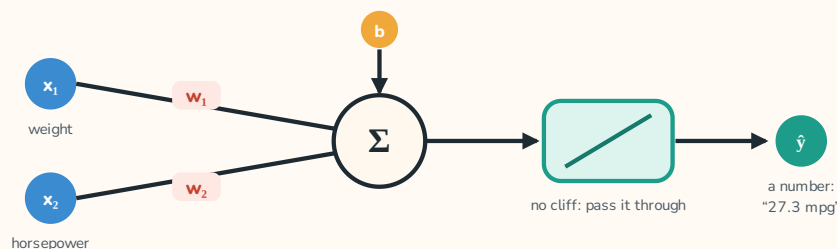
So don’t throw it away. Let the output **be** z . That is the whole change:

$$\hat{y} = z = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b$$

$\hat{y}$, “y-hat”, means “our guess for y ”. In vector shorthand the same line is $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$: the dot product of the knobs with the inputs, plus the bias. Chapter 3 shows how a whole table of cars fits in one such line.



Cliff versus ramp. The perceptron squashes its total into 0 or 1. The regression neuron passes the total straight through: the total *is* the answer. Nothing else about the neuron changes.



The regression neuron. Same inputs, same knobs, same sum as Book 2. Only the last box has changed: it no longer decides, it reports.

IN ONE LINE A **regression** neuron answers “how much?”: its output is the weighted sum itself, a plain number that can be 3.2 or −40 or 1,000.

What the knobs mean now

With one input, $\hat{y} = w \cdot x + b$ is the equation of a **straight line** through a cloud of points: w is the slope (how much $\hat{y}$ goes up when x goes up by 1) and b is where the line crosses the axis. With two inputs, $\hat{y} = w_1x_1 + w_2x_2 + b$ is a flat tilted sheet in three dimensions. With more inputs, a flat sheet in more dimensions. In Book 2 the line was a *wall* that separated two sides. Now the line is a *ruler* that reads off a number. Same line, new job.

Prediction is reading the ruler: put in the car’s numbers, get out a mileage. Once the knobs are set, that is all the neuron ever does — one multiplication per input and one addition. The whole difficulty of this book is *setting the knobs*.

∞ THINK OF IT LIKE... A TAXI METER

A taxi fare is *base charge + rate × distance*. That is $\hat{y} = b + w \cdot x$ with x = kilometres. The meter does not decide anything; it reports a number. Setting the knobs is what the city’s transport office does when it fixes the base charge and the per-kilometre rate — and it does so by looking at data: what trips actually cost drivers. Our neuron will set its own base charge and rate, from examples.

💡 STOP AND THINK

Book 1 said: look at the answer to tell the task. For each of these, is the output a number (regression) or a category (Book 4)? (a) The temperature tomorrow. (b) Will it rain tomorrow? (c) Delivery time for a pizza. (d) Is this review positive? (e) How many likes will this post get?

✓ OUR ANSWER

(a) number · (b) category · (c) number · (d) category · (e) number. Notice (b) and (a) are about the same thing — rain — but one asks “how much/likely” and one asks “yes or no”. The question decides the neuron, not the topic.

⚠ CAREFUL — THE WORDS “LINEAR” AND “NO HIDDEN LAYERS”

Your syllabus calls this a *linear neural network with no hidden layers*. That is a grand name for one neuron with a ramp. “Linear” means the output is a weighted sum of the inputs — a line, a sheet — with no bending. “No hidden layers” means nothing sits between the inputs and the output. We will earn the bends and the layers in Book 5. For now, straight and simple, on purpose.

2

The data: 392 cars

Ingredient one. A famous little dataset from the years when the world suddenly cared about fuel.

★ SPARK — WHY THIS DATASET EXISTS

In 1973 oil prices quadrupled almost overnight. Petrol queues stretched round the block, and for the first time American car buyers asked the question Asha's uncle is asking: *how many miles per gallon?* Magazines began measuring it for every car. The **Auto MPG** dataset, kept at the University of California, Irvine, is a slice of those measurements: 398 cars from 1970 to 1982. It has been used to teach regression for over thirty years, and now it teaches you.

What one row looks like

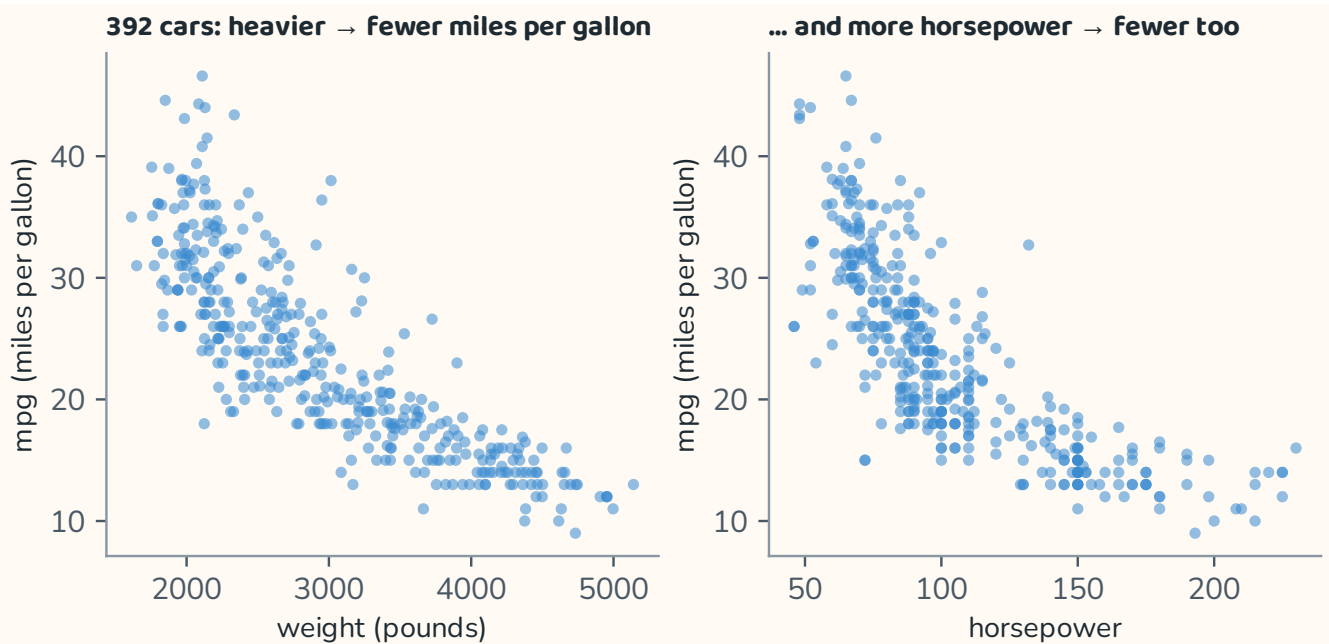
Each car is one example: some measurements (the inputs) and its mileage (the label). Here are four of the 398, exactly as they appear in the file:

mpg	cylinders	displacement	horsepower	weight	acceleration	year	name
18.0	8	307	130	3504	12.0	70	chevrolet chevelle malibu
15.0	8	350	165	3693	11.5	70	buick skylark 320
33.5	4	85	70	1945	16.8	77	datson f-10 hatchback
44.0	4	97	52	2130	24.6	82	vw pickup

Units are American: **mpg** is miles per gallon (1 mpg $\approx$ 0.43 km per litre, so 25 mpg $\approx$ 10.6 km/l), **weight** is in pounds (3,504 lb $\approx$ 1,590 kg), **displacement** is engine size in cubic inches. Six cars have a “?” where horsepower should be; we drop them and keep 392.

Picking the inputs

We will use two inputs: **weight** and **horsepower**. Not because the others are useless, but because two is enough to learn every idea in this book, and few enough that we can still draw pictures. The label is **mpg**. So each car becomes a tiny vector of two numbers going in, and one number coming out — exactly the shape of the neuron in Chapter 1.



Look before you compute. Every dot is a car. Heavier cars and more powerful cars get fewer miles per gallon, and the trend is roughly a straight line going down. That “roughly a straight line” is what makes a one-neuron model a reasonable first try. (It is not perfectly straight — hold that thought for Book 5.)

TRY IT YOURSELF — FIT BY EYE

Lay a ruler across the left-hand picture so it follows the cloud as well as you can. Read off two numbers: where your line is at 2,000 lb and at 4,000 lb. From those, how many mpg does each extra 1,000 lb cost? Write your answer in the margin. In Chapter 6 the neuron will give its own answer; see how close you got.

Hide some cars

Book 1’s rule: never let the model see all the examples. We shuffle the 392 cars and keep **79 of them hidden** (about 20%). The neuron trains on the other **313**. At the end, the hidden cars tell us whether the neuron learned the pattern or merely memorised the training cars. Everything we report as “the real score” comes from the 79 it never saw.

CAREFUL — THE NUMBERS ARE ON DIFFERENT RULERS

Weight runs from about 1,600 to 5,100. Horsepower runs from about 46 to 230. Mpg runs from 9 to 47. The neuron does not know which is which; it only sees numbers, and it will multiply each by a knob. A knob on weight will have to be a thousand times more delicate than a knob on horsepower. That turns out to make gradient descent painfully slow, for a reason we can draw (Chapter 6). The fix is one line — put every input on the same ruler — and we will apply it when we get there. Until then, the four-car example in the next chapters uses friendly numbers on purpose.

WORDS TO KNOW

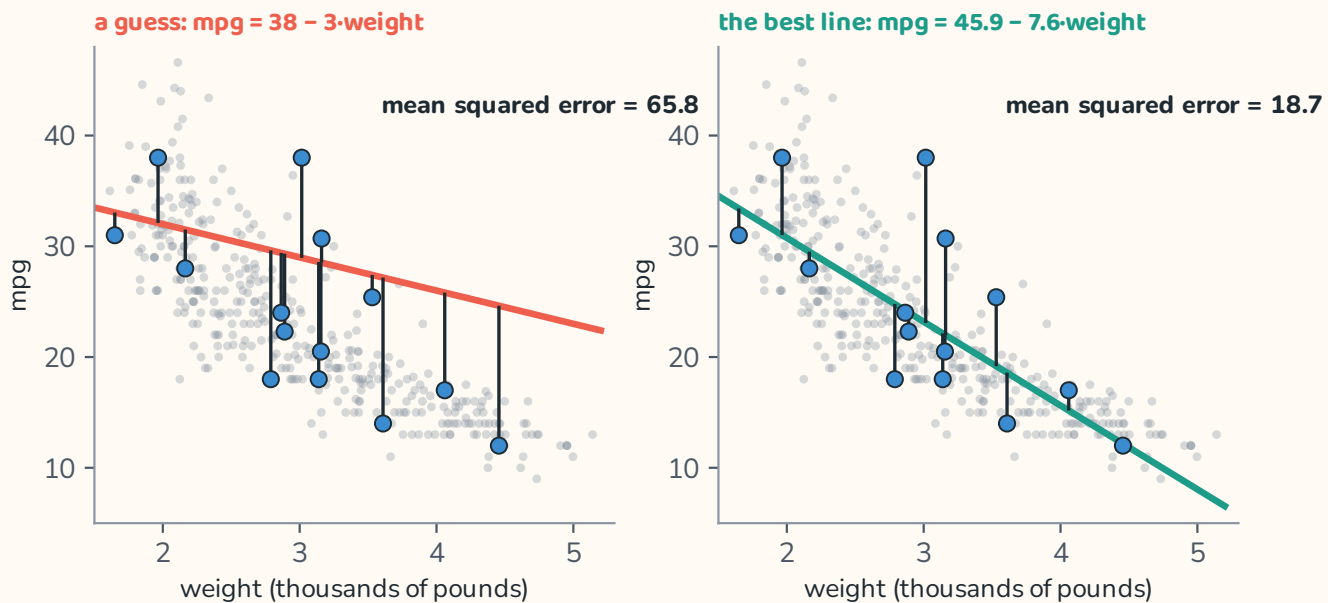
Feature / input — a measurement fed into the model (weight, horsepower). **Label / target** — the correct answer we are trying to predict (mpg). **Training set** — the 313 cars the neuron learns from. **Test set** — the 79 hidden cars used only to score it. **Example** — one row: inputs plus label.

3

The score: how wrong is a line?

Ingredient three. One number that says how badly a line fits the cars — and the surprising shape that number makes.

Draw any line through the cloud of cars. Some cars sit above it, some below. For each car, the gap between the line's guess $\hat{y}$ and the true mpg y is that car's **error**: $e = \hat{y} - y$. A good line has small gaps; a bad line has big ones. We need to turn all those gaps into *one number*, because — as Book 1 said — the machine can only chase one number.



Gaps as sticks. Each vertical stick is one car's error. Left: a guessed line; the sticks are long and mostly point the same way. Right: the best line; the sticks are short and point both ways. The score below each picture is the average of the squared stick lengths.

💡 THE MEAN SQUARED ERROR (MSE)

Square every error, then take the average over all N cars:

1 $\hat{y}_i = w x_i + b$

Guess for car i : read the line at its input.

2 $e_i = \hat{y}_i - y_i$

Error: guess minus truth, sign and all.

3
$$L = \frac{1}{N} \sum_{i=1}^N e_i^2 = \frac{e_1^2 + e_2^2 + \dots + e_N^2}{N}$$

Score: square each error, add them all, divide by how many. One number.

Why square? Two reasons from Book 1: it kills the minus sign (a guess 4 too high and 4 too low are equally wrong, and should not cancel out in the average), and it punishes big misses much more than small ones (a miss of 10 costs 100; a miss of 2 costs 4). **Why average?** So the number means the same thing whether you have 4 cars or 4 million.

Four cars, by hand

For the next three chapters we will use four made-up cars with friendly numbers. The input x is the car's weight in tonnes *minus* 2, so $x = -1$ is a 1-tonne car and $x = +1$ is a 3-tonne car. (Centring the numbers like this keeps the arithmetic clean; we do the same to the real data in Chapter 6.) The label y is mileage.

car	weight	x (weight - 2)	y (mileage)
A	1 t	-1	32
B	1 t	-1	28
C	3 t	+1	12
D	3 t	+1	8

The model is a line, $\hat{y} = wx + b$. Let's score a few candidate lines. For each, compute $\hat{y}$ for the four cars, the four errors, square them, and average.

Try the first row yourself before reading the table: with $w = 0$ and $b = 0$ the line guesses 0 mpg for every car.

line (w, b)	guesses $\hat{y}$ for A, B, C, D	errors $\hat{y} - y$	squared errors, summed	MSE
$w = 0, b = 0$	0, 0, 0, 0	-32, -28, -12, -8	$1024 + 784 + 144 + 64 = 2016$	504
$w = 0, b = 20$	20, 20, 20, 20	-12, -8, +8, +12	$144 + 64 + 64 + 144 = 416$	104
$w = -5, b = 10$	15, 15, 5, 5	-17, -13, -7, -3	$289 + 169 + 49 + 9 = 516$	129
$w = -12, b = 20$	32, 32, 8, 8	0, +4, -4, 0	$0 + 16 + 16 + 0 = 32$	8
$w = -10, b = 22$	32, 32, 12, 12	0, +4, 0, +4	$0 + 16 + 0 + 16 = 32$	8
$w = -10, b = 20$	30, 30, 10, 10	-2, +2, -2, +2	$4 + 4 + 4 + 4 = 16$	4 ← the best

💡 MATRIX FORM — the last row of the table, all four cars in one line each

Stack the four inputs into a column $\mathbf{X}$ and the four labels into a column $\mathbf{y}$. Then “guess for every car” is one multiplication, “error for every car” is one subtraction, and “sum of squares” is one dot product of the error column with itself. This is exactly what the computer does — there is no loop over cars anywhere.

$$1 \quad \hat{\mathbf{y}} = \mathbf{X}w + b = \begin{bmatrix} -1 \\ -1 \\ 1 \\ 1 \end{bmatrix} (-10) + 20 = \begin{bmatrix} 30 \\ 30 \\ 10 \\ 10 \end{bmatrix}$$

Four guesses at once. The bias b is added to every row.

$$2 \quad \mathbf{e} = \hat{\mathbf{y}} - \mathbf{y} = \begin{bmatrix} 30 \\ 30 \\ 10 \\ 10 \end{bmatrix} - \begin{bmatrix} 32 \\ 28 \\ 12 \\ 8 \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \\ -2 \\ 2 \end{bmatrix}$$

Four errors at once — the same numbers as the last row of the table.

3

$$L = \frac{1}{N} \mathbf{e}^T \mathbf{e} = \frac{1}{4} \begin{bmatrix} -2 & 2 & -2 & 2 \end{bmatrix} \begin{bmatrix} -2 \\ 2 \\ -2 \\ 2 \end{bmatrix} = \frac{16}{4} = 4$$

$\mathbf{e}^T \mathbf{e}$ is “each error times itself, added up” — the sum of squares. Written $\|\mathbf{e}\|^2$, the squared *length* of the error column.

With d inputs, $\mathbf{X}$ has d columns and w becomes a column $\mathbf{w}$ of d knobs: $\hat{y} = \mathbf{X}\mathbf{w} + b$. Nothing else changes. Chapter 4 finds the slope of L in this form too, and Chapter 7 shows it as one line of code.

The flat line at 20 (the average mileage) already beats “guess zero” by a mile. Tilting it helps more. The best line is $\hat{y} = 20 - 10x$: it passes exactly between A and B, and exactly between C and D, missing each by 2. Its loss, 4, is the **floor**: no line can do better, because A and B have the same x but different y , so no single line can hit both.

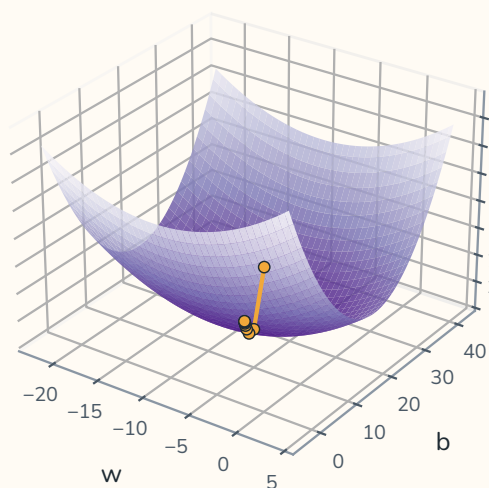
⚠ CAREFUL — THE UNITS OF THE SCORE

Because we squared the errors, the MSE is in *squared* units: mpg². A loss of 4 does not mean “off by 4 mpg”; it means “off by $\sqrt{4} = 2$ mpg, typically”. Take the square root of an MSE (engineers call it the **RMSE**) whenever you want a number you can feel. The neuron itself never needs the square root — it only needs to know which way is *down*.

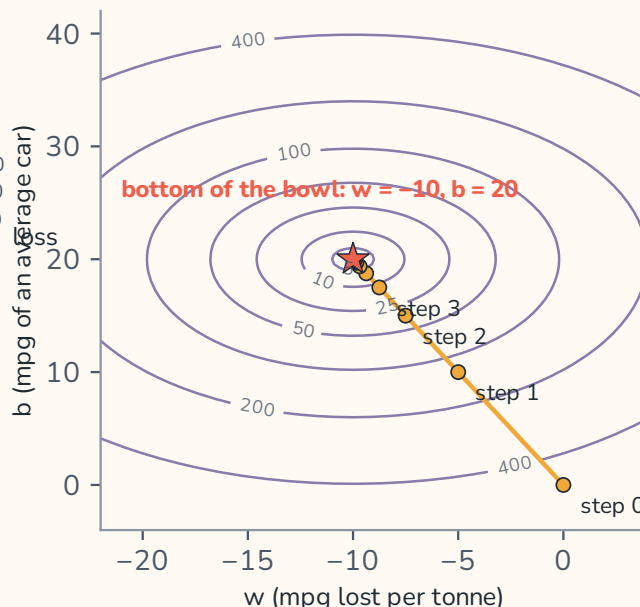
The shape of the score

Here is the idea that turns fitting into walking. The score L depends on the two knobs, w and b . Imagine trying every pair (w, b) — not six lines but a million — and recording the loss for each. What shape do you get?

the loss is a bowl over the two knobs



seen from above: the path walks straight to the bottom



The loss is a bowl. Left: the loss of every (w, b) pair for the four cars, as a surface. Right: the same bowl seen from above, as contour rings (each ring is a loss level; the numbers say which). The bottom is at $w = -10$, $b = 20$ — the best line from the table. The gold dots are the path we will walk in Chapter 5.

It is a bowl, and for this model it is *always* a bowl — one bottom, no bumps, no traps. (The reason: each error is a straight-line function of the knobs, and the square of a straight line is a U. Add U-shapes and you get a U-shape.) So the training problem has become a walking problem: **start anywhere on the bowl and go downhill until you reach the bottom.**

💡 STOP AND THINK

You are standing on the bowl at $w = 0$, $b = 0$, in fog. You cannot see the bottom. What is the *one* thing you can feel that tells you which way to step?

✓ OUR ANSWER

The **slope under your feet**. If the ground rises as you move w up, move w down. If it falls as you move b up, move b up. You need two numbers — the slope in the w direction and the slope in the b direction — and the next chapter works out exactly what they are.

📖 WORDS TO KNOW

Error / residual — $e = \hat{y} - y$ for one example; signed. **Mean squared error (MSE)** — the average of e^2 over all examples; our loss L . **RMSE** — $\sqrt{\text{MSE}}$, a typical miss in the original units. **Loss landscape** — the loss drawn over all possible knob settings; here, a bowl. **Minimum** — the bottom of the bowl: the knobs with the smallest loss.

4

The slope under your feet

If you nudge a knob a tiny bit, how much does the loss change? We work it out from scratch, one line at a time — first for one car with plain numbers, then for every car at once as a matrix — and get the formula that drives all of deep learning.

Stand on the bowl at some (w, b) . Nudge w by a tiny amount δ (say 0.001). The loss changes by some tiny amount. The ratio *change in loss* $\div \delta$ is the **slope in the w direction**. Mathematicians write it $\partial L / \partial w$ and call it a derivative; think of it as “loss-change per unit of knob-turn”. There is a second slope, $\partial L / \partial b$, for the other knob. Together they tell you which way is uphill. Let’s compute them — no formulas from memory, just arithmetic.

PLAIN MATHS — one car, one knob, one nudge

Take a single car with input x and label y , and hold b fixed. Its loss is just its squared error.

1 $e = wx + b - y$

The error of this car at the current knobs.

2 $e_{\text{new}} = (w + \delta)x + b - y = e + \delta x$

Nudge the knob. The error moves by δx . A car with a big x feels the nudge more — its input *amplifies* the knob.

3 $e_{\text{new}}^2 = (e + \delta x)^2 = e^2 + 2ex\delta + x^2\delta^2$

New loss = new error squared. Multiply it out.

4 $e_{\text{new}}^2 - e^2 = 2ex\delta + \underbrace{x^2\delta^2}_{\text{tiny} \times \text{tiny}} \approx 2ex\delta$

Change in loss = new minus old. For $\delta = 0.001$, $\delta^2 = 0.000001$: drop it.

5 $\frac{\partial L}{\partial w} = \frac{\text{change}}{\delta} = 2ex \quad \frac{\partial L}{\partial b} = 2e$

Slope = change $\div \delta$. For b the nudge moves the error by δ itself (no x), so the slope is $2e$.

Try it with numbers. Car A: $x = -1$, $y = 32$, knobs $w = 0$, $b = 0$. The error is $e = 0 \cdot (-1) + 0 - 32 = -32$. So $\partial L / \partial w = 2 \cdot (-32) \cdot (-1) = +64$: turning w up makes the loss go up, 64 per unit. And $\partial L / \partial b = 2 \cdot (-32) = -64$: turning b up makes the loss go down. Both make sense: car A wants a higher guess (it is at 32 and we guessed 0), and since its x is negative, a higher guess means a *more negative* w .

THINK OF IT LIKE... A CHAIN OF TWO LINKS

The knob w does not touch the loss directly. It touches the *error* (link 1: turn w by δ , the error moves by $x\delta$), and the error touches the *loss* (link 2: move the error a little, the squared error moves by $2e$ times that). The slope is the two links multiplied:

$$\frac{\partial L}{\partial w} = \underbrace{\frac{\partial L}{\partial e}}_{\text{link 2: loss per error}} \cdot \underbrace{\frac{\partial e}{\partial w}}_{\text{link 1: error per knob}} = 2e \cdot x$$

This “multiply the links” idea has a name — the **chain rule** — and in Book 5 the chain will be five links long. Nothing new will be needed; just more multiplying.

All the cars

The loss for N cars is the average of the N squared errors. Nudging w changes each car's squared error by its own $2 e_i x_i \delta$. The average changes by the average of those. Here it is first as a sum over cars, then as one matrix product — the two say the same thing.

💡 **PLAIN MATHS** — add up every car's share

$$6 \quad \frac{\partial L}{\partial w} = \frac{2}{N} (e_1 x_1 + e_2 x_2 + \dots + e_N x_N) = \frac{2}{N} \sum_{i=1}^N e_i x_i \quad \text{Error} \times \text{input, averaged over the cars, times 2.}$$

$$7 \quad \frac{\partial L}{\partial b} = \frac{2}{N} (e_1 + e_2 + \dots + e_N) = \frac{2}{N} \sum_{i=1}^N e_i \quad \text{Error} \times 1, \text{ averaged — the bias's "input" is always 1.}$$

$$8 \quad \frac{\partial L}{\partial w_j} = \frac{2}{N} \sum_{i=1}^N e_i x_{ij} \quad \text{Several inputs: knob } w_j \text{ gets the same formula with its own input column } x_{ij} \text{ and the same errors.}$$

The four cars at $(w, b) = (0, 0)$. All guesses are 0, so the errors are $e = (-32, -28, -12, -8)$ and the inputs are $x = (-1, -1, +1, +1)$:

$$\begin{aligned} \frac{\partial L}{\partial w} &= \frac{2}{4} [(-32)(-1) + (-28)(-1) + (-12)(1) + (-8)(1)] = \frac{2}{4} [32 + 28 - 12 - 8] = \frac{2}{4} \cdot 40 = 20 \\ \frac{\partial L}{\partial b} &= \frac{2}{4} [-32 - 28 - 12 - 8] = \frac{2}{4} \cdot (-80) = -40 \end{aligned}$$

↓ the same sums, written as **one matrix product** — this is the line the computer runs

💡 **MATRIX FORM** — every car and every knob at once

Stack the cars as the rows of $\mathbf{X}$ (N rows, one column per input) and the errors as a column $\mathbf{e}$. A sum of "error $\times$ input" over all cars is exactly a dot product of an input column with the error column — and $\mathbf{X}^T$ has the input columns as its rows, so one product does every knob:

$$1 \quad \mathbf{e} = \mathbf{X}\mathbf{w} + \mathbf{b} - \mathbf{y} \quad (N \times 1) \quad \text{Every car's error: guess column minus label column (Chapter 3).}$$

$$2 \quad \nabla_{\mathbf{w}} L = \frac{2}{N} \mathbf{X}^T \mathbf{e} \quad (d \times 1) \quad \text{Row } j \text{ of } \mathbf{X}^T \text{ is input column } j; \text{ dotted with } \mathbf{e} \text{ it gives } \sum_i e_i x_{ij} \text{ — step 8, for every } j \text{ at once.}$$

$$3 \quad \frac{\partial L}{\partial b} = \frac{2}{N} \mathbf{1}^T \mathbf{e} = \frac{2}{N} \sum_i e_i \quad \mathbf{1} \text{ is a column of ones: dotting with it just adds the errors up — step 7.}$$

The same four cars, the same numbers — as one product each:

$$\nabla_w L = \frac{2}{4} \mathbf{X}^T \mathbf{e} = \frac{2}{4} \begin{bmatrix} -1 & -1 & 1 & 1 \end{bmatrix} \begin{bmatrix} -32 \\ -28 \\ -12 \\ -8 \end{bmatrix} = \frac{2}{4} (32 + 28 - 12 - 8) = \frac{2}{4} \cdot 40 = \mathbf{20}$$

$$\frac{\partial L}{\partial b} = \frac{2}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} -32 \\ -28 \\ -12 \\ -8 \end{bmatrix} = \frac{2}{4} \cdot (-80) = \mathbf{-40}$$

In the notebook this is `dw = (2/n) * (X.T @ err)` and `db = (2/n) * np.sum(err)`. Read the code with the matrices next to it and there is nothing left to decode.

Read the two numbers like a compass. $\partial L / \partial w = +20$: the bowl rises in the $+w$ direction, so go the other way — *decrease* w . $\partial L / \partial b = -40$: the bowl falls in the $+b$ direction — *increase* b . And the sizes say the b direction is steeper: b should move more. That matches the picture in Chapter 3: from $(0, 0)$ the bottom is at $(-10, 20)$ — w must fall by 10 and b must rise by 20.

💡 BIG IDEA — THE GRADIENT

The list of all the slopes, one per knob — here $(\partial L / \partial w, \partial L / \partial b) = (20, -40)$ — is called the **gradient**, written ∇L . It always points *uphill*, in the direction where the loss grows fastest. So the direction to walk is **minus the gradient**. That one sentence is the whole of the next chapter.

🔧 TRY IT YOURSELF

Compute the gradient at the flat line $w = 0$, $b = 20$ (the second row of the table in Chapter 3). The errors are $(-12, -8, +8, +12)$. Do it once as a sum and once as $\mathbf{X}^T \mathbf{e}$. Which knob wants to move, and which is already happy?

Check upside down:

$$\begin{aligned} \text{perfect} \\ \text{dL/db} = (2/4)(-12 - 8 + 8 + 12) = 0 &\leftarrow \text{b is already} \\ \text{dL/dw} = (2/4)(12 + 8 + 8 + 12) = 20 &\leftarrow \text{decrease w;} \end{aligned}$$

⚠️ CAREFUL — ABOUT THAT 2

Some books write the loss as $\frac{1}{2} \text{mean}(e^2)$ so that the 2 cancels and the gradient is simply $\text{mean}(e x)$. Others drop the $1/N$ and use a sum. None of this changes the *direction* of the gradient, only its length — and the length gets multiplied by the learning rate anyway. Do not be thrown by formulas that differ by a constant.

🌟 SPARK — COMPARE WITH BOOK 2

Rosenblatt's nudge was $\text{error} \times \text{input}$, with the error forced to be $+1$ or -1 . Look at step 6: the gradient is also $\text{error} \times \text{input}$, averaged — but the error is now a real number. A car missed by 32 pushes sixteen times harder than a car missed by 2. The perceptron rule was gradient descent with its ears blocked. We have just unblocked them.

5

Walking downhill: gradient descent

Ingredient four. Feel the slope, step the other way, repeat. We do it by hand on four cars and watch the loss fall $504 \rightarrow 129 \rightarrow 35 \rightarrow 12 \rightarrow \dots$

THE UPDATE RULE

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

Move each knob *against* its slope (the minus sign), by an amount proportional to the slope (steep $\rightarrow$ big move, gentle $\rightarrow$ small move), scaled by the **learning rate** η , a small number you choose. That is the entire method. Its name is **gradient descent**: go down the gradient.

With many inputs, all the weight knobs move together, and the rule is one vector line — the whole of Chapter 4 folded in:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L = \mathbf{w} - \eta \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} + b - \mathbf{y}) \quad b \leftarrow b - \eta \frac{2}{N} \mathbf{1}^T (\mathbf{X}\mathbf{w} + b - \mathbf{y})$$

guess ($\mathbf{X}\mathbf{w} + b$), error (minus $\mathbf{y}$), slope ($\mathbf{X}^T$ times the error, scaled), step (subtract η times the slope) — the four ingredients of Book 1 in one line

The algorithm, batch style

0 • Start	Set all knobs to 0 (or small random numbers). Choose η .
1 • Guess	For every training car, $\hat{y}_i = w x_i + b$ — as a matrix, $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$.
2 • Measure	Every error $e_i = \hat{y}_i - y_i$, and the loss $L = \frac{1}{N} \sum_i e_i^2$ so you can watch it — $\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y}$, $L = \frac{1}{N} \mathbf{e}^T \mathbf{e}$.
3 • Slope	$\frac{\partial L}{\partial w} = \frac{2}{N} \sum_i e_i x_i$, $\frac{\partial L}{\partial b} = \frac{2}{N} \sum_i e_i$ — $\nabla_{\mathbf{w}} L = \frac{2}{N} \mathbf{X}^T \mathbf{e}$, $\frac{\partial L}{\partial b} = \frac{2}{N} \mathbf{1}^T \mathbf{e}$ (Chapter 4).
4 • Step	$w \leftarrow w - \eta \frac{\partial L}{\partial w}$, $b \leftarrow b - \eta \frac{\partial L}{\partial b}$ — $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L$.
5 • Repeat	Go back to 1. Each round is one epoch . Stop after a fixed number of epochs, or when the loss stops falling.

The word **batch** means step 3 looks at *all* the cars before taking one step. Book 2 nudged after every single example; Book 4 will do that again (and something in between). Batch is the calm, careful version: one look at everything, one step.

By hand: four cars, $\eta = 0.25$

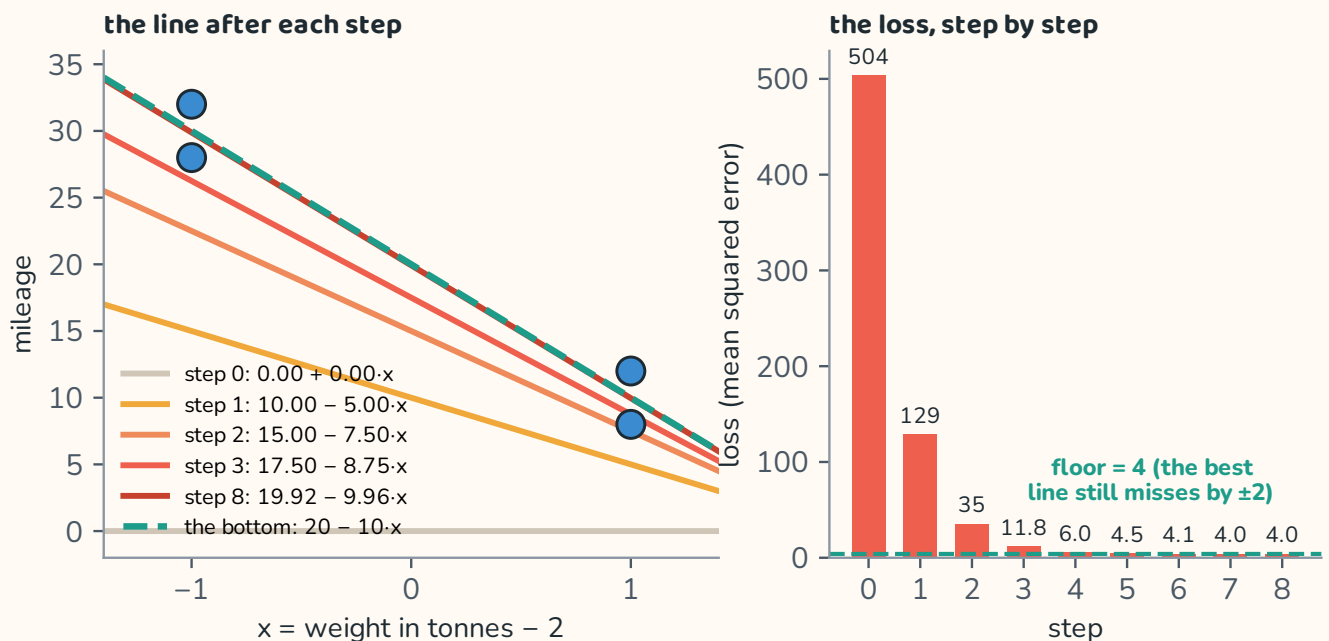
Start at $w = 0$, $b = 0$. We already know the gradient there: $(20, -40)$. So the first step is

$$w \leftarrow 0 - 0.25 \cdot 20 = -5 \quad b \leftarrow 0 - 0.25 \cdot (-40) = 10$$

Here are the first steps in full, then the pattern.

step	w	b	guesses $\hat{y}$ for A, B, C, D	errors $\hat{y} - y$	loss	dL/dw	dL/db	new w	new b
0	0	0	0, 0, 0, 0	-32, -28, -12, -8	504	20	-40	-5	10
1	-5	10	15, 15, 5, 5	-17, -13, -7, -3	129	10	-20	-7.5	15
2	-7.5	15	22.5, 22.5, 7.5, 7.5	-9.5, -5.5, -4.5, -0.5	35.25	5	-10	-8.75	17.5
3	-8.75	17.5	26.25, 26.25, 8.75, 8.75	-5.75, -1.75, -3.25, +0.75	11.81	2.5	-5	-9.375	18.75
4	-9.375	18.75	...	...	5.95	1.25	-2.5	-9.6875	19.375
...	→ -10	→ 20			→ 4	→ 0	→ 0		

How each cell is made — check step 1 yourself, both ways. As a sum: $\sum_i e_i x_i = (-17)(-1) + (-13)(-1) + (-7)(1) + (-3)(1) = 17 + 13 - 7 - 3 = 20$, so $\partial L / \partial w = \frac{2}{4} \cdot 20 = 10$. As a product: $\mathbf{X}^T \mathbf{e} = [-1 \ -1 \ 1 \ 1] [-17 \ -13 \ -7 \ -3]^T = 20$, the same. Then $w \leftarrow -5 - 0.25 \cdot 10 = -7.5$. ✓



Watching the walk. Left: the line after steps 0, 1, 2, 3 and 8, closing in on the best line (dashed). Right: the loss at each step — a fast fall that slows as it nears the floor of 4. Compare the heights of the bars with the loss column in the table.

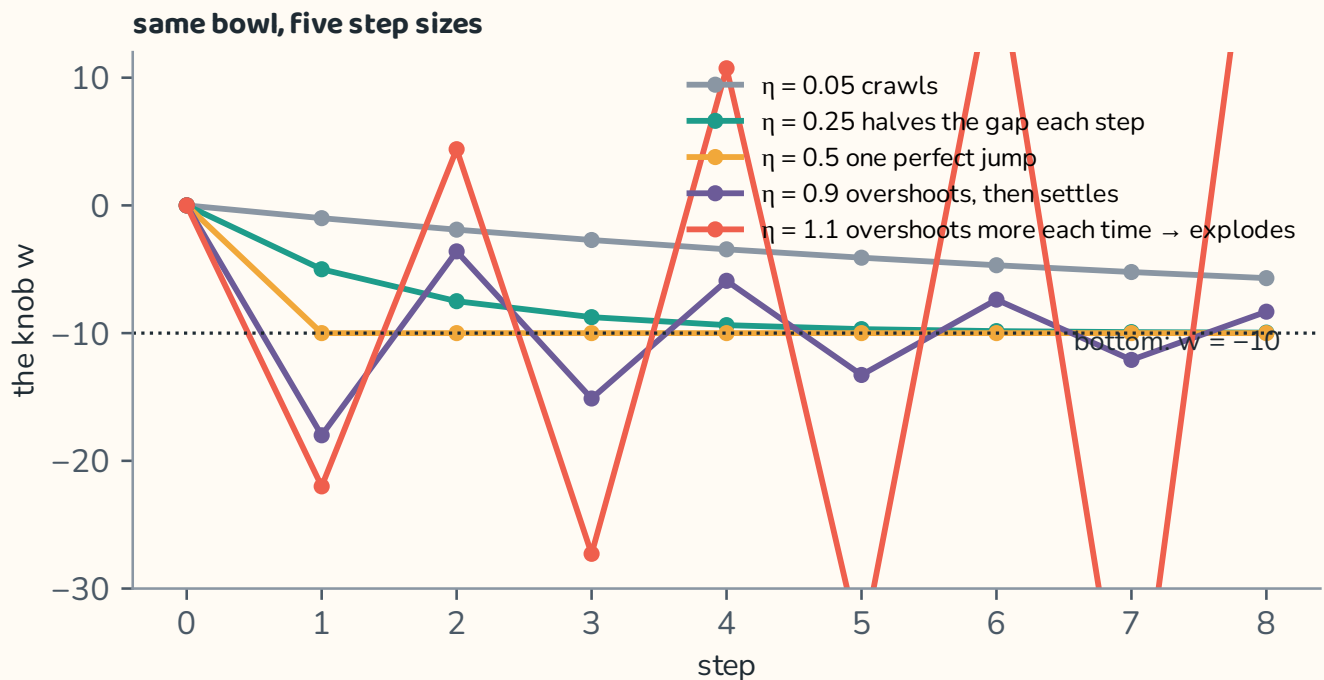
Why every step closes exactly half the gap

Look at the b column: 0, 10, 15, 17.5, 18.75 ... Each step goes halfway from where it is to 20. The w column does the same towards -10 . This is not luck; with these four cars it can be shown in three lines. Because the x values average to zero, the b -slope simplifies to $\partial L / \partial b = \frac{2}{N} \sum_i e_i = 2(b - 20)$, where 20 is the average mileage. So

- $b_{\text{new}} = b - \eta \cdot 2(b - 20) = b - 0.5(b - 20)$ the update, with $\eta = 0.25$ so $2\eta = 0.5$
- $b_{\text{new}} - 20 = (b - 20) - 0.5(b - 20) = 0.5(b - 20)$ subtract 20 from both sides and group
- $w_{\text{new}} + 10 = 0.5(w + 10)$ the same algebra for w , whose bottom is at -10

The gap to the bottom is multiplied by 0.5 every step. In general the multiplier is $1 - 2\eta$: it is 0.5 because we chose $\eta = 0.25$. Change η and you change the multiplier — which brings us to the most important knob you will ever set by hand.

The learning rate: five step sizes on the same bowl



Same bowl, five learning rates. Each curve tracks the knob w over eight steps. $\eta = 0.05$ creeps (multiplier 0.9). $\eta = 0.25$ halves the gap (0.5). $\eta = 0.5$ lands on the bottom in one jump (multiplier 0). $\eta = 0.9$ overshoots to the far side and bounces back, shrinking (-0.8). $\eta = 1.1$ overshoots *more* each time (-1.2) and flies out of the bowl. Only this toy bowl has such a clean rule; real bowls are found by trying — but the five behaviours are universal.

∞ THINK OF IT LIKE... WALKING DOWN A VALLEY IN THE DARK

Small steps: safe, slow, you will get there eventually. Bigger steps: faster. Too big: you step clean over the valley floor and land on the opposite slope, higher than before, and the next step throws you further still. There is no formula for the right step in general. You watch the loss curve — falling smoothly means good; crawling means too small; jumping around or growing means too big — and you adjust. Every deep learning engineer does this, every day.

💡 STOP AND THINK

Book 2's perceptron stopped when it made zero mistakes. When should gradient descent stop? Notice that on the four cars the loss never reaches zero — the floor is 4.

✓ OUR ANSWER

Not at zero loss — that may not exist. Stop when the loss *stops improving* (the gradient is nearly 0, the steps are tiny), or after a fixed budget of epochs, or — as we will see in Book 4 — when the loss on the hidden test set starts to rise. “Good enough” is now a legal answer. That is the gift of a smooth loss.

Training versus using. After six steps the knobs are $w \approx -9.84$, $b \approx 19.69$. Training is over; freeze them. To *predict* for a new 2.5-tonne car, compute $x = 2.5 - 2 = 0.5$ and read the line: $\hat{y} = 19.69 - 9.84 \cdot 0.5 \approx 14.8$ mpg. No gradient, no loss, no label needed — just the ruler. Book 1 called this *inference*. It is the cheap part.

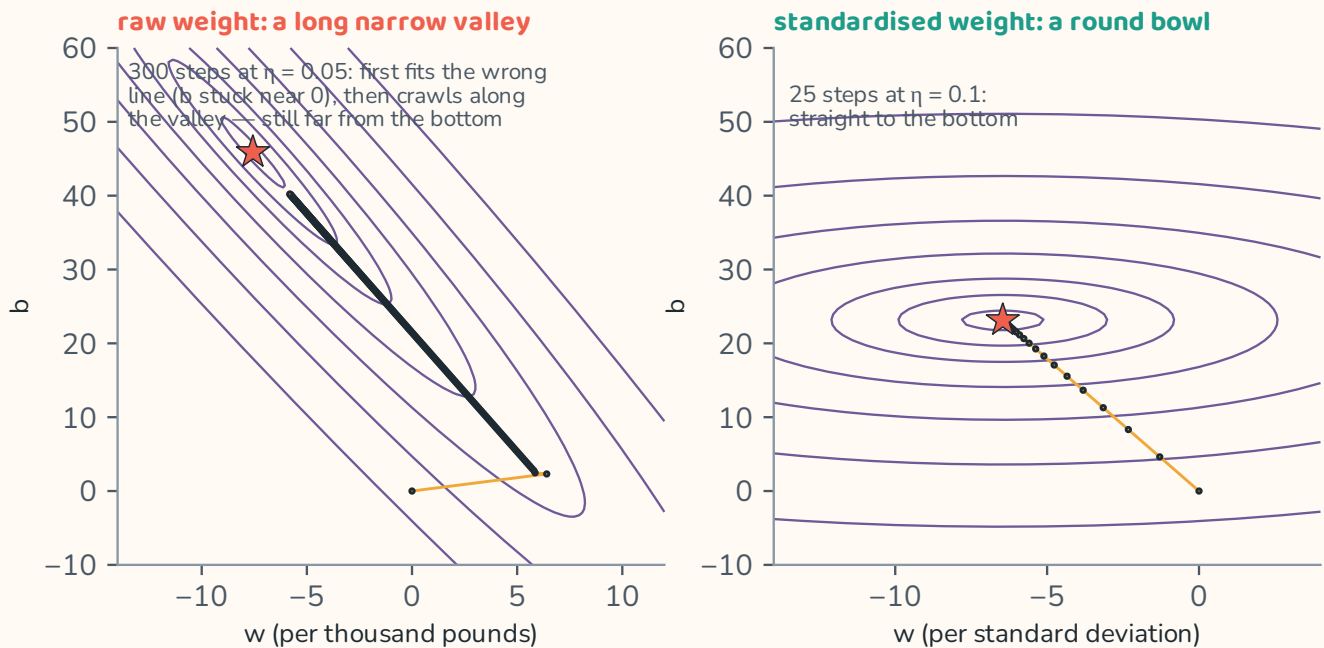
6

The real thing: 392 cars

Same neuron, same loss, same walk — on real data. One preparation step first, and it is the one most beginners skip.

Put every input on the same ruler

In the four-car example we used “weight in tonnes minus 2”, and the walk was clean. Chapter 2 warned that raw weight (thousands of pounds) and raw horsepower (tens) live on very different rulers. Here is what that does to the bowl:



Why we scale. Left: with raw weight as the input, the bowl is a long, thin, tilted valley. Gradient descent first races to the nearest valley floor — which is the *wrong* line, with b stuck near 0 — and then crawls along the valley; after 300 steps it is nowhere near the bottom (★). Right: with standardised weight, the bowl is round, and 25 steps go straight to the bottom.

💡 STANDARDISE: SUBTRACT THE MEAN, DIVIDE BY THE SPREAD

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma} \quad \mu = \frac{1}{N} \sum_{i=1}^N x_i \text{ (the mean)}, \quad \sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2} \text{ (the spread)}$$

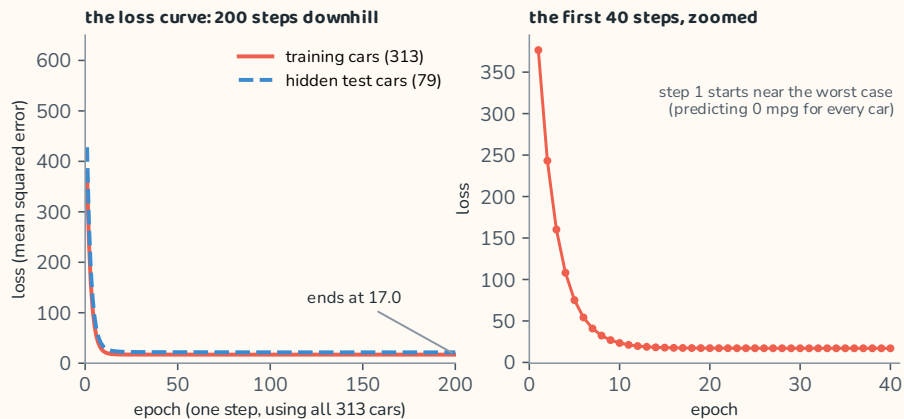
for every column of $\mathbf{X}$ at once: subtract that column's mean, divide by that column's spread — $(X - \mu) / \text{sd}$ in the notebook

Afterwards every input reads as “how many standard deviations away from an average car”: 0 is average, +1 is noticeably heavy, −2 is very light. Every input now has the same size, the bowl is round, and one learning rate suits all the knobs. For our training cars: weight has mean 3,008 lb and spread 854 lb; horsepower has mean 105 and spread 38.

Use the training cars' mean and spread for everything — for the test cars and for every future car. The ruler is part of the model.

Train

Two inputs (scaled weight, scaled horsepower), knobs at zero, $\eta = 0.1$, 200 epochs of batch gradient descent on the 313 training cars. After every epoch we record the loss on the training cars and, separately, on the 79 hidden ones.



The loss curve. Epoch 1 starts at 376 (guessing 0 mpg for everyone is very wrong). By epoch 5 it is 75; by epoch 10, 23; by epoch 50, 17.0 — and then it stays there, because the bottom has been reached. The dashed test curve follows closely: the neuron is learning the pattern, not the particular cars.

epoch	1	5	10	50	100	200
training loss	376.6	75.2	23.4	17.0	17.0	17.0

Read the knobs

$$\hat{y} = 23.13 - 4.63x_{\text{weight}} - 2.13x_{\text{horsepower}} \quad \text{i.e.} \quad \hat{\mathbf{y}} = \mathbf{X} \begin{bmatrix} -4.63 \\ -2.13 \end{bmatrix} + 23.13$$

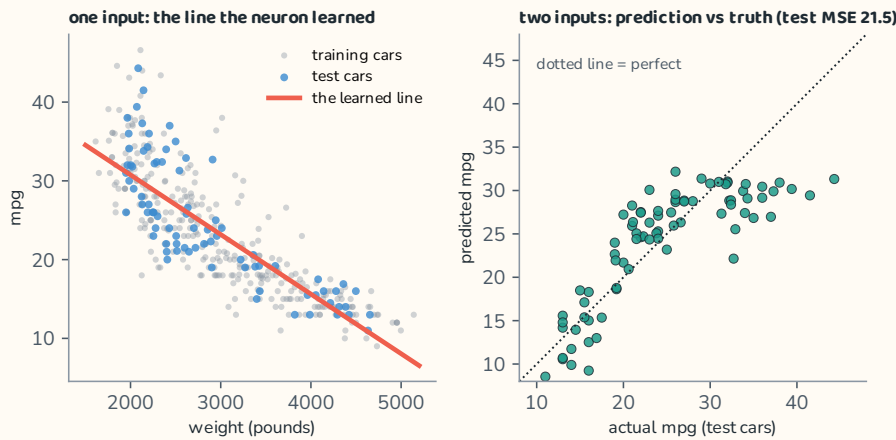
both inputs are the scaled versions; $\mathbf{X}$ is 313 cars $\times$ 2 inputs during training, and one row when a new car walks in

- $b = 23.13$. The guess for a perfectly average car (both scaled inputs 0). It is exactly the average mpg of the training cars — with centred inputs, the bias always learns the average.
- $w_1 = -4.63$. Each extra *standard deviation* of weight (854 lb) costs about 4.6 mpg, holding horsepower fixed. In raw units: about 5.4 mpg per 1,000 lb.
- $w_2 = -2.13$. Each extra 38 hp costs about 2.1 mpg, holding weight fixed. Both signs are negative, as the scatter plots promised.

Score it honestly

model	training loss (313 cars)	test loss (79 hidden cars)	typical miss ($\sqrt{\text{test loss}}$)
always guess the average (23.1 mpg)	59.9	64.6	8.0 mpg
one input: weight	18.2	20.7	4.5 mpg
two inputs: weight + horsepower	17.0	21.5	4.6 mpg

Three things to notice. The neuron beats the no-brain baseline by a lot: a typical miss of 4.6 mpg instead of 8. The training and test losses are close (17 vs 21), so it has not memorised. And adding horsepower to weight helped on the training cars but *not* on the hidden ones — because heavy cars have big engines, the two inputs carry almost the same news, and the second one mostly adds noise. More inputs are not automatically better.



What it learned. Left: with weight alone, the neuron’s line through the cloud — compare it with the line you drew by eye in Chapter 2. Right: for the 79 hidden cars, predicted mpg against actual mpg. Perfect predictions would lie on the dotted diagonal. The misses are larger for the most economical cars, which the straight line under-predicts.

Use it: a new car walks in

Asha’s uncle’s first car weighs 2,500 lb and has 90 hp. Prediction is three lines of arithmetic:

- 1 $x_{\text{weight}} = \frac{2500 - 3008}{854} = -0.595$ $x_{\text{hp}} = \frac{90 - 105}{38.1}$ Scale with the *training* mean and spread.
- 2 $\hat{y} = 23.13 - 4.63(-0.595) - 2.13(-0.393) = 23.13 + :$ Read the ruler. As a matrix:

$$\begin{bmatrix} -0.595 & -0.393 \end{bmatrix} \begin{bmatrix} -4.63 \\ -2.13 \end{bmatrix} + 23.13 -$$

one row of $\mathbf{X}$.
- 3 Say it with its uncertainty: “about 27 mpg, give or take 4 or 5” — because the typical miss on hidden cars was 4.6.

⚠ CAREFUL — A RULER ONLY READS INSIDE ITS MARKINGS

The line says a 5,000 lb car gets about 8 mpg, and a 10,000 lb truck would get *negative* mpg. The model never saw a 10,000 lb vehicle; the straight line simply keeps going. A model is only trustworthy in the region where it had data. Look at the right-hand picture again, too: the very economical cars (35–45 mpg) are all under-predicted. The cloud is slightly *curved* — mileage flattens out for very light cars — and a straight line cannot bend. Bending needs the layers of Book 5.

The demo: a regression neuron in twenty-five lines

Chapters 3 to 6 as code. Every printed number in this book comes from these cells.

The companion notebook `Book3_Regression_From_Scratch.ipynb` loads the cars, runs the four-car example, trains the neuron, draws the loss curve, and predicts for a new car. Only NumPy, pandas (to read the file) and Matplotlib are used. The learning itself is the fifteen lines below.

1 • The neuron, the loss, the gradient, the loop

```
class LinearNeuron:
    #  $\hat{y} = w \cdot x + b$  — no threshold, no squashing: the total IS the answer
    def __init__(self, n_inputs, lr=0.1):
        self.w = np.zeros(n_inputs); self.b = 0.0; self.lr = lr
        self.train_loss, self.test_loss = [], []

    def predict(self, X):
        return X @ self.w + self.b          # one number per car (matrix @ vector)

    def mse(self, y_hat, y):
        return np.mean((y_hat - y)**2)      # the score (Chapter 3)

    def fit(self, X, y, epochs=200, X_test=None, y_test=None):
        n = len(y)
        for _ in range(epochs):
            err = self.predict(X) - y        # 1-2 • guess, then measure: signed error
            dw = (2/n) * (X.T @ err)         # 3 •  $dL/dw_j = (2/n) \sum_i err_i x_{ij}$ 
            db = (2/n) * np.sum(err)         #  $dL/db = (2/n) \sum_i err_i$ 
            self.w = self.w - self.lr * dw    # step downhill, all cars at once = BA
            self.b = self.b - self.lr * db
            self.train_loss.append(self.mse(self.predict(X), y)) # 4 • record, repeat
            if X_test is not None:
                self.test_loss.append(self.mse(self.predict(X_test), y_test))
        return self
```

Two NumPy tricks carry the whole thing. `X @ self.w` multiplies every car's inputs by the weights and adds them up, all cars at once — 313 dot products in one line. `X.T @ err` is $\mathbf{X}^T \mathbf{e} = \sum_i e_i x_{ij}$ for every knob j at once, the whole gradient in one line. That is the matrix rung of Chapter 4, verbatim. There is no hidden library doing the learning; this *is* the learning.

2 • Prepare the data, then train

```
df = pd.read_csv('mpg.csv').dropna(subset=['horsepower']) # 392 cars
idx = np.random.default_rng(0).permutation(len(df)); cut = int(0.8 * len(df))
train, test = df.iloc[idx[:cut]], df.iloc[idx[cut:]] # 313 / 79

feats = ['weight', 'horsepower']
mu, sd = train[feats].mean(), train[feats].std(ddof=0) # TRAINING mean and spread
X_train = ((train[feats] - mu) / sd).values; y_train = train['mpg'].values
X_test = ((test[feats] - mu) / sd).values; y_test = test['mpg'].values

model = LinearNeuron(n_inputs=2, lr=0.1).fit(X_train, y_train, 200, X_test, y_test)
print('w =', model.w, ' b =', model.b)
print('train loss', model.train_loss[-1], ' test loss', model.test_loss[-1])
```

```
w = [-4.633 -2.132] b = 23.129
train loss 16.98 test loss 21.48
```

3 • Predict for a car it has never seen

```
new_car = np.array([[2500.0, 90.0]]) # pounds, horsepower
new_std = (new_car - mu.values) / sd.values # the SAME ruler as in training
print(new_std, model.predict(new_std))
```

```
[[ -0.595 -0.393]] [26.72]
```

4 • What else is in the notebook

Section	What it does	Book chapter
1	The four cars: the loss of a few lines, the gradient-descent trace (step 0 to 6), and the five learning rates.	Chapters 3, 4, 5
2	Loads Auto MPG (downloads it, or uses the local copy), drops the 6 cars with unknown horsepower, draws the scatter plots.	Chapter 2
3	Splits 313 / 79 and standardises with the training mean and spread.	Chapters 2, 6
4	The class above; trains it; prints the knobs; draws the loss curve.	Chapters 4, 5, 6
5	Predicted-vs-actual on the hidden cars; a new car; the one-input line over the cloud.	Chapter 6
6	Six things to try: learning rates that crawl and explode; training without scaling; more inputs; absolute instead of squared error; the exact least-squares formula; the negative-mpg truck.	everything

Look back, look ahead

The whole book in six sentences, a self-check, questions worth carrying, and where Book 4 begins.

The story in six sentences

1. Take the cliff off the perceptron and its output is the weighted sum itself — a **number**. With one input that is a line; with two, a sheet. Reading it is **prediction**.
2. Score a line by the **mean squared error**: average of $(\text{guess} - \text{truth})^2$. Squaring removes the sign and punishes big misses; averaging makes the score independent of N .
3. Drawn over all knob settings, the loss is a **bowl** with one bottom, so training is walking downhill.
4. The slope under your feet is the **gradient**: $\partial L / \partial w = \frac{2}{N} \sum_i e_i x_i$ and $\partial L / \partial b = \frac{2}{N} \sum_i e_i$ — derived by nudging a knob and multiplying two links of a chain; for every knob at once, $\nabla_{\mathbf{w}} L = \frac{2}{N} \mathbf{X}^T \mathbf{e}$.
5. **Gradient descent**: $w \leftarrow w - \eta \partial L / \partial w$, using all examples per step (**batch**); the **learning rate** η decides whether you crawl, converge, bounce or explode.
6. On 392 real cars — after **standardising** the inputs so the bowl is round — the neuron learns 23.1 – 4.6·weight – 2.1·horsepower and misses hidden cars by about 4.6 mpg, against 8 for guessing the average.

Explain it to a friend

Cover the right column. Answer out loud, in your own words. Then compare.

Question	A good short answer
What is the only difference between this neuron and the perceptron?	No threshold at the end. The output is z itself, so it can be any number.
Why square the errors?	To stop positive and negative misses cancelling, and to punish big misses more than small ones.
Why is the loss a bowl?	Each error is a straight-line function of the knobs; a straight line squared is a U; a sum of U's is a U.
Where does the “2·e·x” come from?	Nudge w by δ : the error moves by $x \cdot \delta$ (link 1), the squared error moves by $2e$ times that (link 2). Multiply the links.
Why is there a minus sign in the update?	The gradient points uphill; we want down.
What does “batch” mean?	Use all the training examples to compute one gradient, then take one step.
What goes wrong if η is too big?	You step over the bottom onto the far slope, higher than before; big enough and each step is worse than the last.
Why standardise the inputs?	So every input is on the same scale; otherwise the bowl is a long thin valley and gradient descent crawls.
Why use the training mean and spread for a new car?	The scaling is part of the model; the knobs were learned on that ruler.

✧ QUESTIONS WORTH CARRYING (NO ANSWERS HERE — ON PURPOSE)

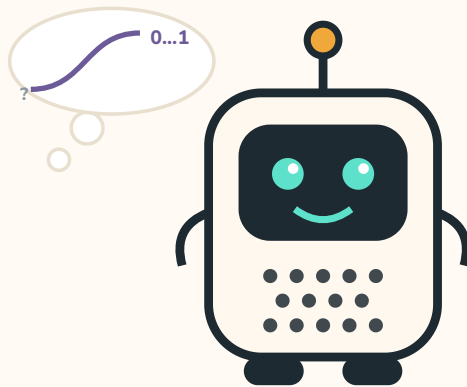
- 1 · The four-car floor was 4 because cars A and B share an input but not a label. What in the 392-car data plays the same role? Could any model, however clever, get the test loss to zero?
- 2 · Gradient descent used all 313 cars for every step. Imagine a million cars. What would you change, and what would you risk?
- 3 · The bias learned the average mpg. Suppose you had forgotten to centre the inputs — would it still? What would b mean then?
- 4 · The perceptron rule was “gradient descent with its ears blocked”. What exactly did the step function block? Write the perceptron’s loss, if it has one.
- 5 · Two links made the chain rule here. Sketch the chain for a network with one hidden neuron between x and $\hat{y}$. How many links now?

✧ SPARK — THERE IS A SHORTCUT, AND WE IGNORED IT

For this one model — a straight line, a squared loss — a formula jumps to the bottom of the bowl in one calculation. It is two hundred years old (Legendre and Gauss, fitting the orbits of comets) and is called *least squares*; notebook Section 6 shows it agrees with gradient descent to three decimals. So why walk when you can jump? Because the moment we add a bend or a second layer (Book 5), the formula vanishes and the walk is all we have. We learned to walk on the one bowl where we could check our footing.

Words to know — the whole book

Regression Predicting a number (“how much”).	Linear neuron $\hat{y} = w \cdot x + b$ with no threshold; a line, or a flat sheet.	Prediction / inference Reading the line for a new input, with frozen knobs.
Error / residual $e = \hat{y} - y$ for one example.	MSE Mean squared error, the loss: $\text{mean}(e^2)$.	RMSE $\sqrt{\text{MSE}}$; a typical miss in the original units.
Derivative dL/dw Loss-change per unit knob-turn: the slope in one direction.	Gradient ∇L All the slopes together; points uphill.	Chain rule Slope of a chain = product of the slopes of its links.
Gradient descent $w \leftarrow w - \eta \cdot \nabla L$, repeated.	Learning rate η The step size; too small crawls, too big explodes.	Batch Using all examples for each gradient.
Epoch One pass through the training data; in batch mode, one step.	Convergence The loss stops falling; the steps become tiny.	Standardise $(x - \text{mean})/\text{std}$, using training statistics.
Baseline Guess the average; the score to beat.	Train / test loss Loss on the cars seen in training / on the hidden cars.	Extrapolation Predicting outside the data’s range; not trustworthy.



End of Book 3

Nova can now walk downhill. It is wondering how to walk towards a yes or a no.

NEXT TIME — BOOK 4: GUESSING “WHICH ONE”

Asha’s uncle bought the car. Now her cousin Meera has a harder question from the hospital: given a few measurements of a tumour, is it harmful or harmless? A yes/no answer — but this time we will not use a cliff. We squash the neuron’s total gently into a number between 0 and 1, read it as “how sure”, and score it by *how surprised* we are when the truth arrives. The bowl comes back, the gradient comes back, the walk comes back.

Then we train on real medical data one example at a time — **stochastic** gradient descent — and, for the question “which of three flowers is this?”, we line up three neurons and let them share out a vote with **softmax**. By the end of Book 4, every ingredient for a real network will be on the table.

Deep Learning, Told Simply · a five-book story for the Deep Learning (DNN) course · Prof. Saurabh